ePolicies AI Assistant — Full Project Notes

These notes explain the whole project from start to end in simple English. They are based on the code in this repository and on the real runs on the company laptop. Anything guessed is marked **(inferred)**; anything made up for illustration is marked **(example)**.

---

1. PROJECT OVERVIEW

What is it? The ePolicies AI Assistant is a question-answering tool for our 235 ePolicies documents. You type a question in plain English, and it answers using **only** those documents, showing the exact policy passages it used as clickable sources. It is a **RAG** system (Retrieval-Augmented Generation: first *retrieve* the right passages, then let the AI *generate* an answer from them).

What problem does it solve?

- Policy information is spread across 235 long documents; finding one rule by hand is slow.
- A normal AI chatbot can make up answers. This tool answers only from our files, checks every citation, and says "not found" when the files do not cover a question.

Input and final output

	What it is
Input (one time)	235 Markdown (`.md`) policy files in the `output` folder
Input (every use)	A question typed in the website (or in the terminal)
Final output	An answer in Markdown with `[S1]`, `[S2]` source markers, a list of source cards (document, section, exact passage, links), a verification status, and warnings for conflicts or missing information

---

2. FOLDER AND FILE STRUCTURE

```
epolicies-rag/
├── .env.example            template for the backend settings (copy to .env)
├── .env                    your private settings (not in Git)
├── output/                 the 235 policy .md files (company data, not in Git)
├── artifacts/              embedding.jsonl backup is written here
├── setup-windows.bat       one-click Windows setup
├── START-HERE.md, README.md, PRESENTATION_NOTES.md, PROJECT_NOTES.md
├── docker-compose.yml      optional PostgreSQL + pgvector in Docker
├── backend/                Python code (the "engine")
│   ├── requirements.txt    Python libraries to install
│   ├── app/
│   │   ├── main.py, cli.py, config.py, setup_tools.py, evaluation.py
│   │   ├── ingestion/      read, split and embed the policy files
│   │   ├── database/       PostgreSQL tables and queries
│   │   ├── retrieval/      search (meaning + keyword + exact) and ranking
│   │   ├── agents/         the AI steps: plan, check evidence, answer, verify
│   │   ├── llm/gemini.py   connection to Google Gemini
│   │   ├── services/rag_service.py   runs all answer steps in order
│   │   ├── api/            web endpoints the website calls
│   │   └── schemas/        shapes of requests and responses
│   ├── tests/              110 automated tests + small invented test files
│   ├── eval/               question sets for measuring accuracy
│   └── scripts/            short helper scripts (wrappers around cli.py)
└── frontend/               Next.js website (what users see)
    ├── app/                pages: chat, documents
    ├── components/         chat window, source cards, sidebar, diagnostics
    └── lib/api.ts          sends questions to the backend
```

Important files, in 1–2 lines each

File	What it does
`backend/app/config.py`	Reads all settings from `.env` (model names, database address, folder, search sizes) and checks they are valid.
`backend/app/cli.py`	The command-line tool: `setup`, `doctor`, `init-db`, `discover`, `check-files`, `ingest`, `resume`, `status`, `ask`, `search`, `evaluate` and more.
`backend/app/setup_tools.py`	The `setup` wizard (writes `.env`) and the `doctor` health check.
`backend/app/main.py`	Starts the FastAPI web server and adds error handling and logging.
`ingestion/markdown_loader.py`	Finds every `.md` file, reads it safely (fixes text encoding), makes a stable ID from its path, and computes a fingerprint (hash). Also safe ZIP import.
`ingestion/markdown_parser.py`	Reads the Markdown structure: title, headings, paragraphs, lists, tables and links.
`ingestion/chunker.py`	Cuts each document into small sections ("chunks") at headings, keeping tables together.

File	What it does
`ingestion/embedder.py`	Sends chunks to `gemini-embedding-001` and checks each result has exactly 768 numbers.
`ingestion/pipeline.py`	Runs the whole indexing process and works out which files are new, changed, unchanged or deleted.
`ingestion/jsonl_store.py`	Writes, checks and reloads the `embedding.jsonl` backup file.
`database/schema.sql`, `schema.py`	Create the database tables; choose pgvector or the built-in fallback.
`database/repository.py`	All database reads and writes (save a document and its chunks in one step, list documents, chat history).
`retrieval/vector_search.py`	"Meaning" search: finds chunks whose embeddings are closest to the question's.
`retrieval/keyword_search.py`	Keyword search (PostgreSQL full-text search) and exact matching of codes like `HR-POL-012`.
`retrieval/ranking.py`	Reciprocal Rank Fusion: merges ranked lists; removes duplicate text.
`retrieval/hybrid_search.py`	Runs all three searches for each query and merges them into one ranking.
`agents/query_planner.py`	Understands the question: finds codes and acronyms, decides if it is simple or complex, rewrites follow-ups.
`agents/retrieval_agent.py`	The search loop: search → check evidence → search again (max 3 rounds).
`agents/evidence_checker.py`	Asks Gemini "do these passages answer the question? what is missing?"
`agents/answer_generator.py`	Asks Gemini to write the answer with `[S#]` markers; has a plain-text fallback.
`agents/citation_verifier.py`	Checks every citation in code, and asks Gemini to double-check weak ones.
`agents/prompts.py`	The instructions given to Gemini, and wraps policy text as "untrusted data".
`llm/gemini.py`	Connects to Gemini (API key or Vertex AI), retries temporary errors, reads JSON answers.
`services/rag_service.py`	Runs every answer step in order and builds the final response.
`api/routes_chat.py`, `routes_documents.py`, `routes_ingestion.py`	Web endpoints: `/chat`, `/chat/stream`, `/documents`, `/search`, `/health`, `/ingest`.
`frontend/components/chat/chat-workspace.tsx`	The chat page: sends the question, shows progress, shows the answer.
`frontend/components/citations/source-card.tsx`	The expandable source card with the exact passage.
`frontend/lib/api.ts`	The website's connection to the backend (normal and streaming requests).

3. FULL FLOW — STEP BY STEP

The project has two phases:

- **Phase A, indexing:** done once, then again when files change. Command: `python -m app.cli ingest`.
- **Phase B, answering:** done for every question.

Phase A — Indexing

Step A1. Find the policy files

- **WHAT:** looks through `MARKDOWN_DIR` (your `output` folder) and its sub-folders for `.md` files.
- **WHY:** the system must know exactly which documents exist.
- **HOW:** `markdown_loader.discover_markdown_files()`. It skips hidden folders and only follows shortcuts that stay inside the folder.
- **EXAMPLE:** `output/` → 235 `SourceFile` records such as `Coordination Of Benefits (COB) - Policy.md`.

Step A2. Read each file and give it an ID and a fingerprint

- **WHAT:** reads the text, fixes the encoding, makes a stable document ID and a content hash.
- **WHY:**
  - The ID stays the same every run, so the same file is never stored twice.
  - The hash ("fingerprint") changes only when the content changes, so unchanged files can be skipped.
- **HOW:** `markdown_loader.load_document()`, `make_document_id()`, `sha256_text()`. If a file is not valid UTF-8 it is read as Windows text (cp1252) with a warning.
- **EXAMPLE (real code output):** `Coordination Of Benefits (COB) - Policy.md` → `doc_7f5beb596584f560184296a0`

Step A3. Decide what changed

- **WHAT:** compares the files on disk with what is already in the database: new, modified, unchanged, deleted, renamed.
- **WHY:** only changed files need work, which saves time and Gemini cost.
- **HOW:** `pipeline.build_plan()`.
- **EXAMPLE:** first run: 235 new; second run with no changes: 235 unchanged and 0 Gemini calls.

Step A4. Read the document structure

- **WHAT:** finds the title, headings, paragraphs, lists, tables and links.
- **WHY:** a split by heading keeps related text together; tables must not be cut in the middle.
- **HOW:** `markdown_parser.parse_markdown()`, using the `markdown-it-py` library.
- **EXAMPLE (from a synthetic test file `hr/leave-policy.md`):** headings found: `Employee Leave Policy > Leave Entitlements`, `Carryover > Exception for Approved Projects`, `Requesting Leave`.

Step A5. Cut into chunks

- **WHAT:** splits each document into sections of about 512 tokens (a token is about 4 characters), at heading boundaries.
- **WHY:** search works on small focused pieces; the AI gets only the relevant pieces.
- **HOW:** `chunker.chunk_document()`.
  - A big table is split into row groups, with the header row repeated in each piece.
  - A long list is split between items; long text is split at sentences, with 64 tokens of overlap.
  - Tiny sections (under 25 tokens) are merged with a neighbour.
- **EXAMPLE (real output on the synthetic test file):** chunk `chk_ae4e21924af86893ca1ed136`: 100 tokens, heading path `Employee Leave Policy > Leave Entitlements`, contains a table.

Step A6. Turn chunks into embeddings

- **WHAT:** each chunk, with its document title and section in front, is sent to `gemini-embedding-001`, which returns 768 numbers.
- **WHY:** these numbers capture meaning, so "reimbursement limit" can match "maximum payment".
- **HOW:** `embedder.GeminiEmbedder.embed_documents()`.
  - It works in batches of 50 chunks, paced at most 60 requests per minute.
  - It retries temporary errors a few times.
  - It checks every result has exactly 768 numbers.
  - Results are saved in the `embedding_cache` table, so identical text is never embedded twice.
- **EXAMPLE:** the text that is embedded looks like this (synthetic test file):

```
Document: Employee Leave Policy (HR-POL-012)
Section: Employee Leave Policy > Leave Entitlements

## Leave Entitlements
| Leave type | Annual entitlement | Requires documentation |
...
```

  → `[0.0123, -0.0345, ...]` (768 numbers) **(example values)**

Step A7. Save to PostgreSQL

- **WHAT:** saves the document row and all its chunks (text, headings, links, embedding, keyword index).
- **WHY:** the database is where every search happens.
- **HOW:** `repository.replace_document()`. Each document is saved in **one transaction** (all or nothing), so a half-saved document is never visible.
- **EXAMPLE:** a row in the `chunks` table: `chunk_id`, `document_id`, `heading_path`, `text`, `embedding`, `search_tsv` (keyword index).

Step A8. Remove deleted files

- **WHAT:** documents whose file disappeared lose their chunks; the document row is kept and marked `deleted`.
- **WHY:** answers must not come from removed policies.
- **HOW:** `repository.mark_document_deleted()`. A safety check refuses to delete everything at once (usually a sign of a wrong folder).

Step A9. Write the backup file

- **WHAT:** writes every chunk and its embedding to `artifacts/embedding.jsonl` (one JSON line per chunk) plus a manifest with a checksum.
- **WHY:** a portable backup; the database can be rebuilt from it with no Gemini calls (`load-jsonl`).
- **HOW:** `jsonl_store.export_jsonl()`. It writes to a temporary file first, then swaps it in, so a half-written file is never used.

Phase B — Answering a question

Step B1. Receive the question

- **WHAT:** the website sends the question to `POST /chat/stream`; the terminal uses `python -m app.cli ask`.
- **HOW:** `routes_chat.chat_stream()` → `rag_service.RAGService.answer()`. Progress messages are streamed to the website ("Searching policies", "Checking evidence"...).

Step B2. Understand the question

- **WHAT:** finds policy codes, acronyms and quoted phrases; decides whether the question is simple or complex.
- **WHY:** codes need exact matching; complex or follow-up questions need a smarter plan.
- **HOW:** `query_planner.plan_query()`. Simple questions use the rule-based `deterministic_plan()` (no AI call). Gemini plans only for follow-ups or complex questions (words like "compare", "exception", "all policies").

Step B3. Search (hybrid)

- **WHAT:** for each planned query, three searches:
  1. **Meaning search:** compare the question's embedding with every chunk's embedding (`vector_search()`).
  2. **Keyword search:** PostgreSQL full-text search on the important words (`keyword_search()`).
  3. **Exact search:** whole-word match of codes like `HR-POL-019` (`exact_search()`).
- **WHY:** meaning search finds reworded answers; keyword and exact search catch precise terms.
- **HOW:** `hybrid_search.hybrid_search()` merges the lists with `ranking.reciprocal_rank_fusion()` (score = 1/(60 + rank) added up over the lists) and removes duplicate text. It keeps the top 8 chunks.

Step B4. Check the evidence (the "agentic" loop)

- **WHAT:** Gemini reads the found passages and says whether they answer the question, what is missing, and new search queries to try.
- **WHY:** one search is often not enough; this lets the system look again on its own.
- **HOW:** `retrieval_agent.RetrievalAgent.run()` + `evidence_checker.assess_evidence()`. It stops when the evidence is sufficient, after 3 rounds, or when a new search finds nothing new.

Step B5. Write the answer

- **WHAT:** up to 12 passages, labelled S1, S2... are given to Gemini with strict rules:
  - use only these sources and cite every statement;
  - keep exceptions;
  - report conflicts and missing information.
- **HOW:** `answer_generator.generate_answer()`. If Gemini's structured (JSON) answer is broken or cut off, it retries, then asks for plain text (`answer_from_markdown()`).

Step B6. Verify the citations

- **WHAT:** code checks that each `[S#]`:
  - points to a passage that was really retrieved;
  - still exists in the database, in the right document;
  - contains the claim's words and every number in it.
- **HOW:** `citation_verifier.verify_deterministic()`. Weak claims go to Gemini (`verify_with_llm()`), and problems trigger one rewrite. With **strict grounding** on, an answer with no verified source is replaced by "I could not find information about this in the indexed policy documents".

Step B7. Build the response and save the chat

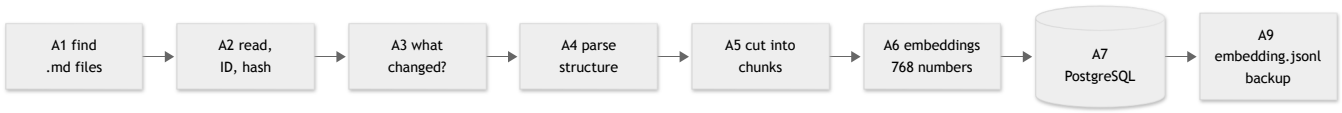
- **HOW:** `RAGService._build_response()` renumbers the sources S1..Sn in order of use and builds the source cards. `_finish()` saves the question and answer in the `conversations` and `messages` tables and adds diagnostics if asked.

Step B8. Show it in the website

- **HOW:** `assistant-message.tsx` renders the Markdown answer; the `[S1]` markers become clickable chips that open `source-card.tsx` with the exact passage.

Flow diagram

Phase A — indexing (`python -m app.cli ingest`)



Phase B — answering (every question; searches the PostgreSQL data from Phase A)



---

4. END-TO-END WALKTHROUGH WITH ONE EXAMPLE QUESTION

Question (real, asked on the company laptop): *"What is the policy on coordination of benefits?"*

Where the data below comes from:

- **(real run):** seen on the laptop screen;
- **(real code output):** I ran that project function on the same question;
- **(example):** typical shape, real values not recorded;
- **(inferred):** a reasoned guess.

Before the question: the documents were indexed (Phase A)

- Input: the 235 files in `output/`, including `Coordination Of Benefits (COB) - Policy.md` and `Coordination of Benefits (COB) - Medicare HRP Claim Processing.md` **(real run)**.
- Output: 10,503 chunks in PostgreSQL. For COB, sections such as *COB Benefit Credit* > *COB Benefit Credit: Overview*, *Overview* > *Types of Plans We Do and Don't Coordinate Benefits With*, *COB Types* > *Maintenance of Benefits* are separate chunks (these heading paths appeared in the real sources).

Step 1 — Question received

- Input: `"What is the policy on coordination of benefits?"`
- Code: `cli.cmd_ask()` → `RAGService.answer()` (terminal) or `routes_chat.chat_stream()` (website).
- Output: a `ChatRequest` object `{question: "...", include_diagnostics: true}`.

Step 2 — Understand the question (`query_planner.plan_query()`)

- `is_complex()` → `False` (no words like "compare" or "exception") **(real code output)**.
- So the rule-based plan is used, with no Gemini call for planning **(real code output)**:

```
{"search_queries": ["What is the policy on coordination of benefits?"],
 "identifiers": [], "retrieval_emphasis": "balanced", "needs_multiple_documents":
false}
```

- Note: the real run showed `planning: 4406.9 ms`. Since no planning call is made, this time was most likely spent creating the Gemini/Vertex AI connection **(inferred)**.

Step 3 — Hybrid search, round 1 (`hybrid_search.hybrid_search()`)

- Keyword terms (question words like "what" and "policy" removed) **(real code output)**: `["coordination", "benefits"]` → full-text query `coordination | benefits`.
- Meaning search: the question is embedded (768 numbers) and compared with all chunk embeddings.
- Exact search: skipped, because no codes were found.
- Merging with RRF works like this. The numbers below are the real function on example IDs A–D **(example)**:

```
vector list:  A(1) B(2) C(3)        keyword list: B(1) D(2) A(3)
B = 1/(60+2) + 1/(60+1) = 0.03252   ← ranked first: high in both lists
A = 1/(60+1) + 1/(60+3) = 0.03227
D = 0.01613,  C = 0.01587
```

- Output: the top 8 COB chunks with their rankings.

Step 4 — Check evidence, rounds 1–3 (`RetrievalAgent.run()`, `assess_evidence()`)

- Gemini judged the evidence **not yet sufficient**, because "coordination of benefits" covers many pages. New queries were searched in rounds 2 and 3 **(real run: `Rounds: 3 (max_rounds_reached)`)**.
- The exact follow-up queries were not recorded **(example):** "COB order of benefit determination", "COB types maintenance of benefits".
- Output: a pool of COB chunks from the two COB documents.

Step 5 — Write the answer (`generate_answer()`)

- Input: up to 12 passages labelled `S1..Sn` inside `<source>` tags, plus the rules.

- Output: an answer in Markdown with markers, organised under these headings **(real run)**:
  - Types of Plans We Coordinate Benefits With `[S2]`
  - Types of Plans We Don't Coordinate Benefits With `[S3]`
  - COB Benefit Credit `[S1]`
  - Order of Benefit Determination (OBD) `[S4][S5]`
  - COB Types: 100% Allowable (NAIC 120 Model, 1996 NAIC COB Model), Maintenance of Benefits `[S6][S7][S8]`

Step 6 — Verify citations (`verify_deterministic()`)

- All 8 sources passed: each exists, was retrieved, and supports its sentence → `[supported]` on all 8; `Verification: verified` **(real run)**.
- `insufficient evidence: True` **(real run)**: Gemini flagged that one answer cannot cover everything about COB.

Step 7 — Final output (real run, shortened)

```
**Types of Plans We Coordinate Benefits With** [S2]:
* Employer-sponsored group health plans ...
**COB Benefit Credit**
Accumulators track the benefit credit amount [S1]. ...
**Order of Benefit Determination (OBD)**
The NAIC OBD rules are followed to determine primary coverage responsibility [S4]. ...

Sources:
  [S1] Coordination Of Benefits (COB) - Policy › ... › COB Benefit Credit: Overview
[supported]
  [S2] ... › Overview: Types of Plans We Do and Don't Coordinate Benefits With
[supported]
  [S3] ... › Overview: Types of Plans We Do and Don't Coordinate Benefits With
[supported]
  [S4] ... › Overview: Order of Benefit Determination (OBD)
[supported]
  [S5] Coordination of Benefits (COB) - Medicare HRP Claim Processing › ... Background
[supported]
  [S6] ... › COB Types: 100% Allowable (Commercial) - NAIC 120 Model
[supported]
  [S7] ... › COB Types: 100% Allowable (Commercial), 1996 NAIC COB Model
[supported]
  [S8] ... › COB Types: Maintenance of Benefits
[supported]

Verification: verified; insufficient evidence: True
Rounds: 3 (max_rounds_reached); LLM calls: 5
timings (ms): planning 4406.9, retrieval 19709.4, answer 13334.0, verification 1578.3,
total 39016.0
```

In simple words: the system turned the question into a search, found the COB sections by meaning and by the words "coordination" and "benefits", searched twice more because the topic is large, gave the best passages to Gemini, and Gemini wrote an answer that cites each passage. The code then checked all 8 citations before showing the answer. The whole run took about 39 seconds.

The 5 AI calls were most likely 3 evidence checks and 1 answer, plus 1 more (a citation check or a plain-text retry); the exact split was not recorded **(inferred)**.

```
"What is the policy on coordination of benefits?"
        │  plan_query → simple, no AI call
        ▼
 hybrid search ("coordination | benefits" + meaning) ──► top 8 COB chunks
        │  assess_evidence: not enough → search again (rounds 2, 3)
        ▼
 generate_answer (gemini-2.5-flash) ──► answer with [S1]..[S8]
        │  verify_deterministic: 8 of 8 supported
        ▼
 Final answer + 8 source cards  (≈39 s)
```

---

5. IMPORTANT CODE EXPLAINED

Function / class	Simple explanation
`discover_markdown_files()`	Lists every `.md` file in the folder, sorted.
`load_document()`	Reads one file, fixes the encoding, computes its fingerprint.
`build_plan()`	Compares disk and database: new, changed, unchanged, deleted, renamed.
`parse_markdown()`	Breaks a file into headings, paragraphs, lists and tables, and keeps their order.
`chunk_document()`	Groups those parts into ~512-token chunks at headings; splits big tables with the header repeated.
`GeminiEmbedder.embed_documents()` / `embed_query()`	Gets 768-number embeddings for chunks / for a question.
`IngestionPipeline.run()`	Runs the whole indexing; only one run at a time (database lock).
`replace_document()`	Saves a document and all its chunks in one all-or-nothing step.
`export_jsonl()` / `load_jsonl()` / `reconcile()`	Write the backup file / rebuild the database from it / compare the two.
`vector_search()`	Meaning search; uses pgvector if installed, otherwise the built-in numpy fallback.
`keyword_search()` / `exact_search()`	Word search with PostgreSQL full-text search / exact whole-word match of codes.
`reciprocal_rank_fusion()`	Merges ranked lists: items high in several lists win.
`hybrid_search()`	Runs the three searches for every query and merges them.
`plan_query()`	Rule-based plan for simple questions; Gemini plan for follow-ups or complex ones.
`RetrievalAgent.run()`	Search → evidence check → search again, max 3 rounds.

Function / class	Simple explanation
`assess_evidence()`	Gemini says whether the passages are enough and what to search next.
`generate_answer()`	Gemini writes the cited answer; falls back to plain text if needed.
`verify_deterministic()` / `verify_with_llm()`	Code checks citations / Gemini double-checks weak claims.
`RAGService.answer()`	The conductor: runs every answer step and builds the final response.
`GeminiLLM.generate_json()`	Asks Gemini for an answer in a fixed JSON shape; retries if cut off.
`call_with_retries()`	Retries only temporary errors (rate limit, server busy), a limited number of times.
`run_doctor()` / `run_setup()`	Health check of folder, database and Gemini / guided first-time setup.

Technical terms

Term	Meaning
Markdown (`.md`)	A plain-text format with simple marks for headings (`#`), lists (`-`) and tables (`
Chunk	A small section of a document; the unit that is searched.
Token	A piece of a word; about 4 characters.
Embedding	A list of numbers (here 768) that represents the meaning of a text.
Cosine similarity	How close two embeddings point in the same direction (1.0 = same meaning).
Full-text search	PostgreSQL's built-in word search, with word stems ("benefits" also matches "benefit").
Hybrid search	Meaning search + keyword search together.
RRF (Reciprocal Rank Fusion)	Formula to merge ranked lists: score = Σ 1/(60 + rank).
Agentic	The AI decides by itself to search again when evidence is missing, within a limit.
Gemini / Vertex AI	Google's AI models / Google Cloud's service for using them with our company account.
SDK	A ready-made code library; here the official Google Gen AI SDK (`google-genai`).
PostgreSQL / pgvector	The database / an optional add-on for fast meaning search (not on the laptop; the fallback is used).
numpy	A Python library for fast number maths; used for the fallback meaning search.
FastAPI / uvicorn	Python web framework for the backend / the program that runs it.
Next.js / npm	Website framework / tool that installs the website's libraries.
JSON / JSONL	Data format of keys and values / a file with one JSON record per line.
Hash (SHA-256)	A fingerprint of a text; changes if any character changes.
Transaction	A group of database changes that all happen, or none do.
`.env`	A settings file with private values (project, password). Never shared or put in Git.

Term	Meaning
SSE (Server-Sent Events)	How the backend streams progress messages to the website.

---

6. HOW TO RUN THE PROJECT

Full commands are in `START-HERE.md` and the "ePolicies RAG – Run From Start (Windows)" doc. Short version for the company laptop:

One-time setup

1. Install Python 3.12, PostgreSQL 16, Node.js 22, Google Cloud SDK and Git (company AppStore).
2. In the project folder:

```
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
```

3. Create your own database (port 5433):

```
& "C:\Program Files\PostgreSQL\16\bin\initdb.exe" -D "$HOME\epolicies-pgdata" -U
postgres -W -E UTF8 --auth=scram-sha-256
& "C:\Program Files\PostgreSQL\16\bin\pg_ctl.exe" -D "$HOME\epolicies-pgdata" -o "-p
5433" -l "$HOME\epolicies-pgdata\log.txt" start
& "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -h localhost -p 5433 -c
"CREATE DATABASE epolicies;"
```

4. Sign in to Google Cloud: `gcloud auth application-default login`
5. In `.env` set `GOOGLE_GENAI_USE_VERTEXAI=true`, `GOOGLE_CLOUD_PROJECT=hcb-dev-genai-tools`, `GOOGLE_CLOUD_LOCATION=us-central1`, `GEMINI_MODEL=gemini-2.5-flash`, `GEMINI_THINKING_BUDGET=0`, `DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5433/epolicies`, `MARKDOWN_DIR=./output`.

Index and test (in `backend`)

```
python -m app.cli init-db          # expect: vector_backend : array
python -m app.cli doctor           # expect: Summary: 0 problem(s)
python -m app.cli ingest           # expect: documents_indexed : 235, failed_documents : 0
python -m app.cli ask "What is the policy on coordination of benefits?" --diagnostics
```

The last command is the example from section 4. Expect an answer, a `Sources:` list with `[supported]` on each source, and `Verification: verified`.

Daily start

1. Database: the `pg_ctl.exe ... start` line above.
2. Terminal 1 (`backend`): `uvicorn app.main:app --port 8000` → `Uvicorn running on http://127.0.0.1:8000`
3. Terminal 2 (`frontend`): `npm run dev` → `Local: http://localhost:3000`

4. Browser: `http://localhost:3000` (type `http`, not `https`).

---

7. RESULTS

Real results from the company laptop (`ingest` report and the `ask` run):

Result	Value
Files discovered / readable	235 / 235
Empty files / duplicate-content files	0 / 0
Documents indexed / failed	235 / 0
Chunks created	10,503
Embeddings generated / reused	9,321 / 1,182
`embedding.jsonl` records	10,503
Embedding model / dimensions	`gemini-embedding-001` / 768
Answer model	`gemini-2.5-flash` (Vertex AI, project `hcb-dev-genai-tools`, `us-central1`)
COB question	8 sources, all `[supported]`, verified, 3 rounds, 5 AI calls, 39.0 s total
Automated tests	110 passed, on the development Mac

1,182 embeddings were reused even though there were no duplicate files. This is most likely because the same section text appears in more than one document, and identical text is embedded only once **(inferred)**.

The meaning-search fallback took about 1.9 ms per query in a benchmark with 10,503 random vectors on the development Mac. This is a benchmark, not a measurement on the laptop.

Not available in the project:

- **Indexing time:** not recorded. Next run, note the start and end time of `ingest`.
- **Accuracy on real questions:** no real test set yet. The `evaluate` command and `backend/eval/epolicies_eval.template.jsonl` are ready; add about 20 questions with known source files.
- **Time saved and cost per question:** not measured. Time a few manual searches against the tool; token counts are shown in Diagnostics.

---

8. CHALLENGES AND SOLUTIONS

Problems met during setup

Problem	How it was solved
Gemini API key rejected (HTTP 401)	The company provides Gemini through Vertex AI: added `gcloud` login support (`llm/gemini.py`, `make_client()`).

Problem	How it was solved
No pgvector on the laptop (needs admin rights)	Built-in fallback: embeddings stored as plain number arrays, meaning search done with numpy (`schema.py`, `vector_search.py`).
`gemini-3.5-flash` not enabled in our project (404)	Switched to `gemini-2.5-flash` with one setting.
Answers cut off ("malformed JSON")	`GEMINI_THINKING_BUDGET=0`, an automatic retry with more room, and a plain-text fallback.
Database stops after a restart	The daily start command, and `doctor` shows the fix.
Wrong folder or typo in `.env`	`doctor` and the `setup` wizard check each setting and print the fix; a warning appears when a terminal setting overrides `.env`.

Edge cases handled in the code

Edge case	Handling
Files with odd text encoding	Read as UTF-8, otherwise Windows encoding (cp1252), with a warning.
Same file name in different folders	The ID comes from the full relative path, so names never collide.
Duplicate or renamed files	Identical text is embedded once; renames are detected and cost no new embeddings.
Big tables	Split by rows, with the header repeated in every piece.
Interrupted indexing (network, quota)	Each finished batch is saved; `resume` continues without repeating work.
Saving fails halfway	One transaction per document; the old version stays searchable.
Wrong folder makes all files "disappear"	Refuses to delete the whole index unless told to.
Prompt injection inside a policy file	Policy text is wrapped as untrusted data; instructions inside it are ignored.
Made-up or wrong citations	Code checks each one; strict grounding hides unsupported answers.
Temporary Gemini errors (429, 503)	Limited retries with waiting time between them.
Wrong embedding size	Every embedding is checked to have exactly 768 numbers.

---

9. LIMITATIONS AND NEXT STEPS

Limitations

- Runs on one laptop; the database must be started after each restart.
- No user login: chat history is shared by anyone using the app on that machine.
- Very broad questions may be flagged "insufficient evidence"; focused questions work best.
- Accuracy, time saved and cost have not been measured.
- Token counts are estimates (about 4 characters per token).
- Citation checking compares words and numbers. It is strong, but it is not a full proof of meaning, so users should open the source for important decisions.
- Approval for sending policy text to Vertex AI still needs confirming.

Next steps

1. Build a 20-question test set with known answers and run `python -m app.cli evaluate`.
2. Confirm data-use approval with the manager or compliance.
3. Ask IT to install pgvector for the database.
4. If the team finds it useful, move it to a shared server with company sign-on (SSO).

---

10. QUICK SUMMARY (1 minute)

"We have 235 ePolicies documents, and finding one rule means reading through long pages. I built an assistant that answers questions using only those documents. For example, I asked 'What is the policy on coordination of benefits?'. The system searched our policies by meaning and by keywords, looked again twice because COB is a big topic, and then Google's Gemini wrote an answer covering which plans we coordinate with, the COB benefit credit, the order of benefit determination and the COB types. Every sentence links to the exact policy section, and the code checked all eight sources before showing the answer. It took about 39 seconds. All 235 files are indexed with zero failures. If the policies don't cover a question, it says so instead of guessing. Next, I want to measure its accuracy with a set of real questions."

---

11. POSSIBLE QUESTIONS

1. **Does it use the internet or general AI knowledge?** No. It searches only our 235 files. Gemini receives only the question and the matching passages, and answers without a verified source are blocked.
2. **What if the answer isn't in the policies?** It says it could not find the information and shows no sources.
3. **How do I know an answer is right?** Each sentence has a source marker that opens the exact passage, and the code checked every citation. For important decisions, read the source.
4. **How accurate is it?** Not measured yet. The evaluation tool is ready; it needs a set of real questions with known answers.
5. **How fast is it?** About 39 seconds for the COB question (3 search rounds). Other questions have not been timed.
6. **What happens when policies change?** Run `python -m app.cli ingest`; only new or changed files are processed.
7. **Is the data safe?** The files stay on the laptop. Only passages go to Gemini in our company Vertex AI project, using the company login with no key stored in code. Data approval should be confirmed.
8. **What does it cost?** Not measured. 9,321 chunks were embedded once; each question makes a few Gemini calls (5 for the COB example).
9. **Why not just use Ctrl+F?** Ctrl+F misses reworded answers and cannot combine several sections into one answer with sources.
10. **Can the whole team use it?** Not yet; it runs on one laptop. A shared server with company sign-on is a next step.